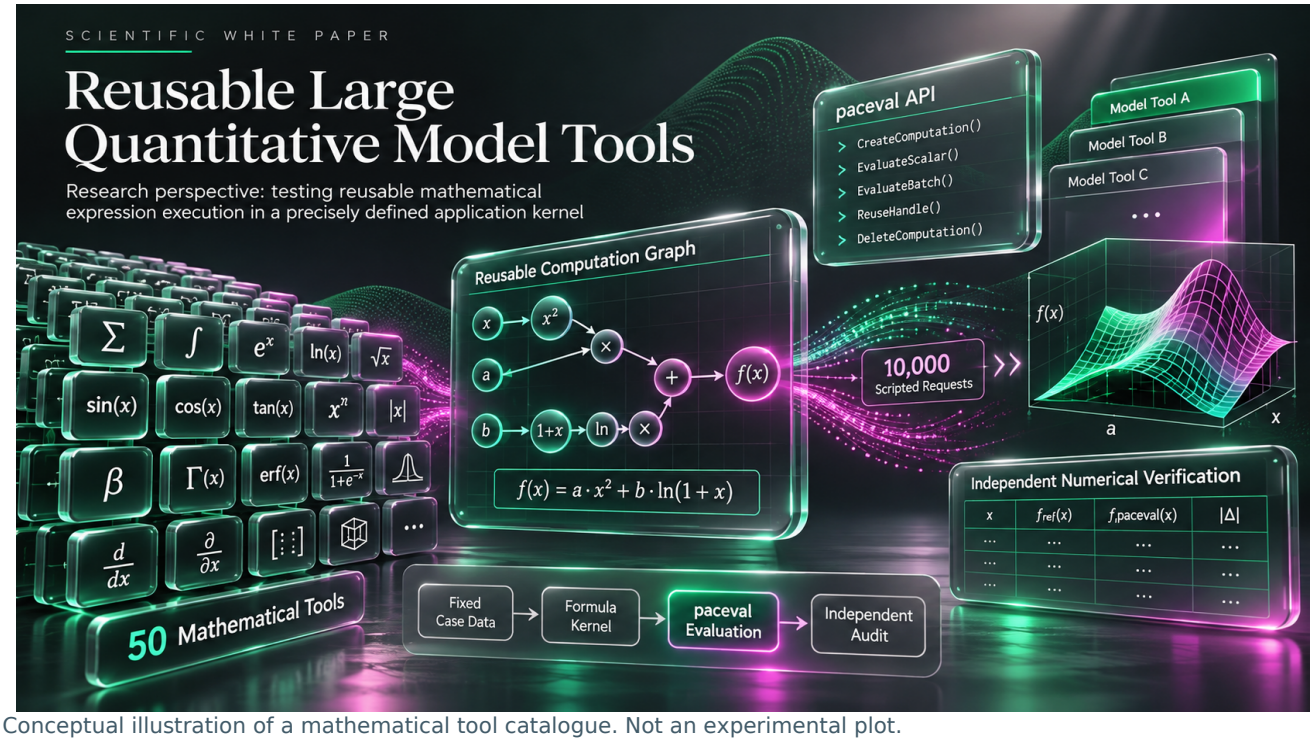
Reusable Large Quantitative Model Tools

Fifty mathematical tool definitions, persistent paceval computation graphs, deterministic invocation and independently audited composition.



Conceptual illustration of a mathematical tool catalogue. Not an experimental plot.

Recorded research outcome	Value
Unique tool definitions	50 across five mathematical families
Persistent paceval handles	51 (including an immutable formula revision)
Scripted requests	10,000
Numerical reference	Native C++ and an independent 80-digit sample audit

Evidence type: actual local C++/paceval SDK execution on synthetic dimensionless mathematical models. No claim is made about a live MCP endpoint, independent AI reasoning, or industrial deployment.

1. Introduction: Industrial use case and current state of the art

Industrial analytics platforms increasingly expose narrowly defined computational capabilities to applications and language-model agents. A manufacturing engineering assistant, for example, may need several nonlinear response calculations in a fixed sequence; the required mathematical functions should be deterministic, individually versioned, and independently testable. The key engineering challenge is preserving mathematical contracts while tools are repeatedly invoked and recombined.

The Model Context Protocol (MCP) provides a standard interface for describing and invoking tools. Its 28 July 2026 release introduces a stateless protocol core and updated authorization capabilities; the current tools specification describes named tools, JSON Schema input/output declarations and structured results. These are protocol-level concepts, distinct from the numerical execution layer examined here.

Research boundary

This experiment implements the local C++ mathematical execution layer using the official paceval facade. Fifty function definitions persist as computation graphs and can be invoked repeatedly by a deterministic dispatcher. The work neither runs a network MCP server nor evaluates the quality of a language model selecting the correct tools. Tool authorization, request routing, JSON Schema validation and user consent remain future system concerns.

References: MCP 2026-07-28 specification and release; paceval public API; NIST AI Risk Management Framework. Full URLs appear on the final page.

2. Scientific research question and execution boundary

Can a diverse catalogue of quantitative formula tools be represented as persistent paceval mathematical expressions, executed across many scripted requests and composed deterministically, while retaining native and independent numerical agreement?

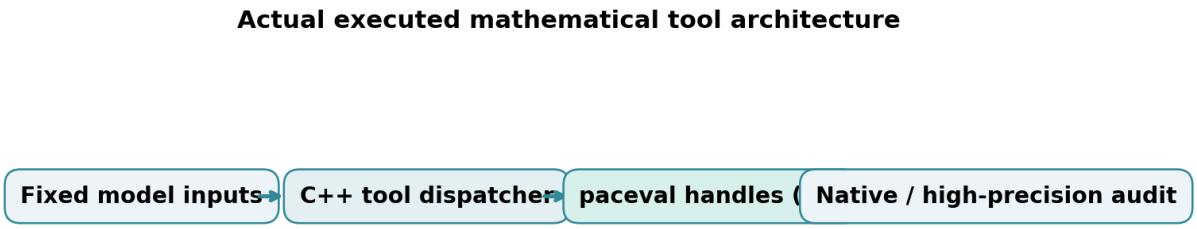


Figure 1. The implemented boundary: host orchestration and validation; paceval expression graphs; independently implemented reference.

Separation of responsibilities

Executed by paceval	Executed by C++ / Python host
Graph creation and handle reuse	Tool registration, request dispatch, domain checking
Scalar double-precision formula evaluation	Synthetic dataset construction and composition routing
Packed batch formula evaluation	CSV export, native reference, 80-digit sample audit

The dispatcher does not interpret arbitrary untrusted expressions. Each formula is frozen before execution. A formula update creates a new computation object and retains the original version.

3. Mathematical tool definitions

Each model takes five dimensionless inputs x, y, a, b and c. The frozen experimental domain is x in [0.1,2.9], y in [-1,1] and a,b,c in [0.2,1.4]. Each of five distinct families has ten coefficient values k_j = 0.11 + 0.07j, j = 0,...,9.

Five independently implemented formula families

A: f = a*x^2 + b*ln(1+x) + k*c*y

B: f = a*exp(-k*x) + b*sin(x+y) + c*x*y

C: f = a/(1+k*x^2) + b*sqrt(1+x) + c*cos(y)

D: f = (a*x+b*y+c)/(1+k*x^2) + sin(x*y)

E: f = a*ln(1+x^2) + b*exp(-x*y^2) + k*c*x^2

These expressions provide polynomial, logarithmic, exponential, trigonometric, rational and square-root operations, with distinct nonlinear composition. The paceval strings use ln(); the independent C++ reference uses std::log(). All denominators and special-function domains are defined over the admitted input set.

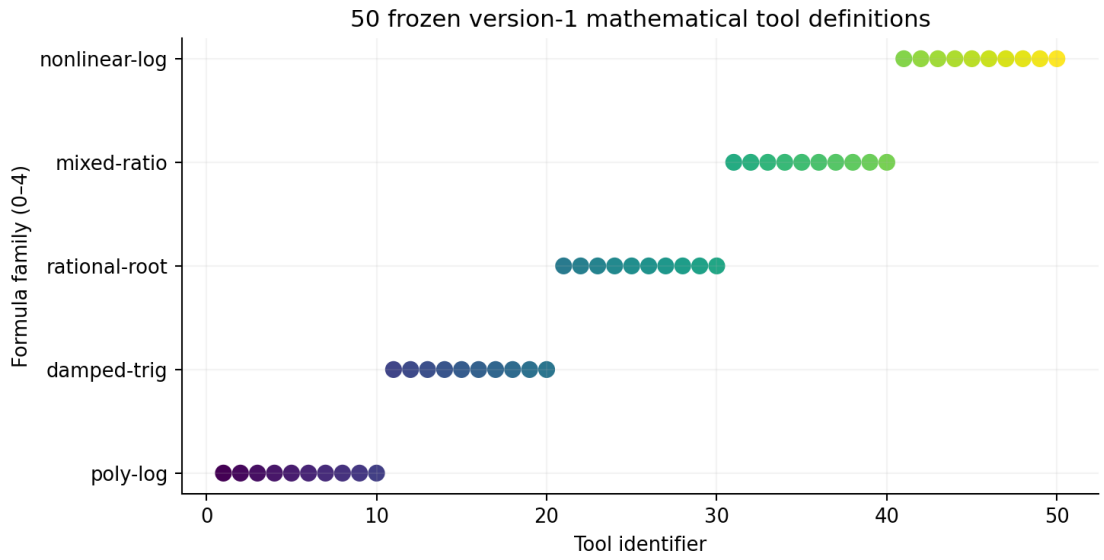


Figure 4. Version-1 registry of fifty expressions, categorized into five non-equivalent mathematical families.

4. Workload and immutable mathematical versions

A deterministic xorshift-derived generator produces 10,000 scripted local requests assigned cyclically to fifty tools. Every fifth request is followed by another actual paceval formula invocation. The host supplies tanh(first_output) as the second function's y input, producing 2,000 deterministic two-stage compositions. This is model composition, not a demonstration of autonomous tool discovery.

```
u = f_i(x,y,a,b,c); v = f_j(x,tanh(u),a,b,c)
```

The model registry records each exact formula, its coefficient and an immutable version identifier. The demonstration creates a new version-2 formula for tool 1 while retaining the original graph. The version comparison evaluates both on identical inputs and independently checks the modified result.

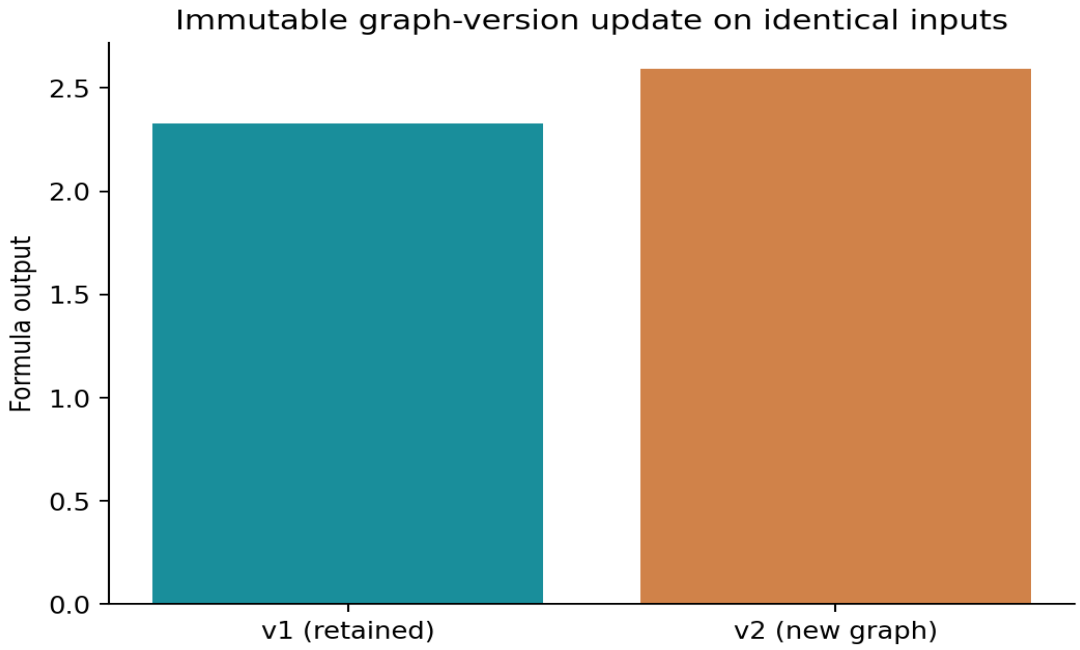


Figure 7. Same inputs evaluated by preserved version 1 and independently created version 2 of a mathematical tool.

The updated version is an explicit change of formula definition, not an in-place mutation of a cached calculation handle.

5. Actual SDK execution and independent numerical audit

Verified quantity	Recorded value
Distinct frozen mathematical tools	50
Persistent graph handles	51
Scripted requests	10,000
Two-stage compositions	2,000
Scalar paceval evaluations	15,723
Batch-comparison samples	1,600
3D surface evaluation points	3,721
Maximum absolute difference vs native C++	1.7763568394002505e-15
Independent 80-digit checks	250 / 250 passed
CMake/CTest	1 / 1 passed

The C++ program creates each paceval handle once; each request supplies fresh numerical arguments to the same computation. Batch checks pack thirty-two independent input records per graph and compare all results with a separately coded C++ reference. The numeric audit rejects nonfinite values and discrepancies exceeding 1e-10*(1+|reference|).

A separate mpmath-based program independently reimplements the five families at 80-digit precision and checks every fortieth exported request. Its maximum sampled absolute discrepancy was about 2.07e-15. Neither the finite native reference nor this finite multiprecision sample establishes a global error bound.

6. Mathematical diversity and numerical accuracy

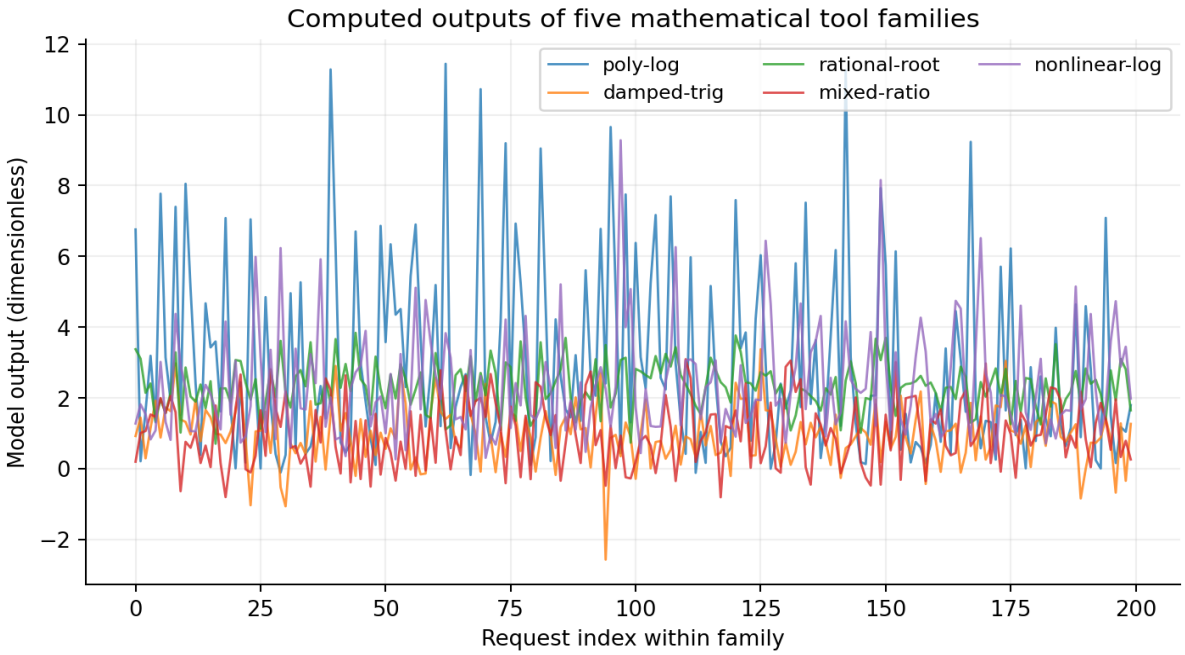


Figure 2. Recorded outputs of five mathematically distinct tool families on deterministic input sequences.

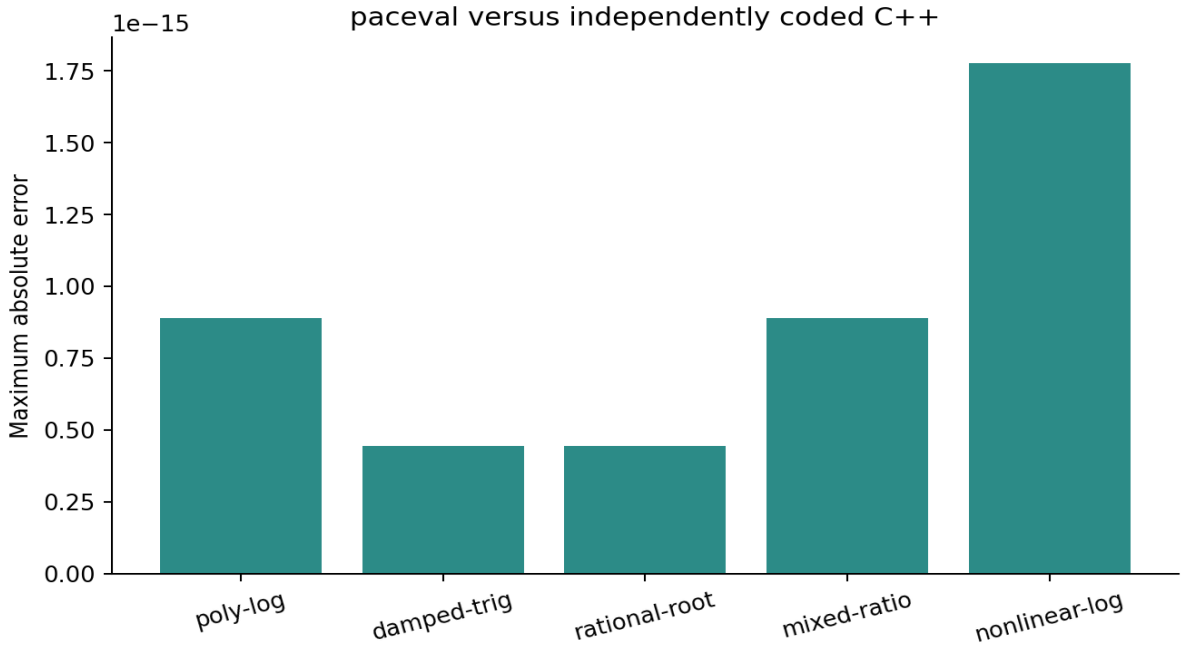


Figure 3. Maximum observed absolute difference between actual paceval results and the independently implemented native equations, grouped by family.

The plotted discrepancies characterize the executed input samples only. They are not evidence of performance superiority or a proof of correctness for arbitrary function strings.

7. Three-dimensional mathematical response visualization

A separately recorded parameter sweep evaluates tool 23 on a 61 x 61 mesh, keeping a=0.9, b=0.8 and c=0.7 fixed while varying x and y across the declared input range. Each displayed height is an actual paceval computation result saved to response_surface.csv.

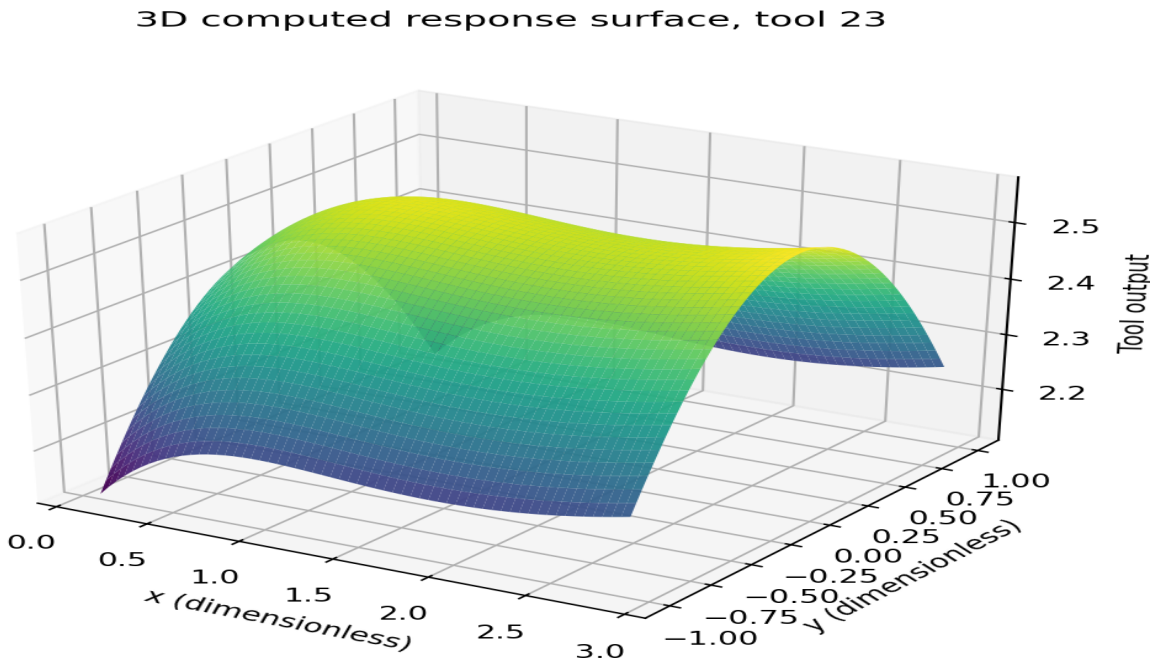


Figure 5. 3D mathematical response surface: 3,721 real paceval evaluation values, not AI-generated surface geometry.

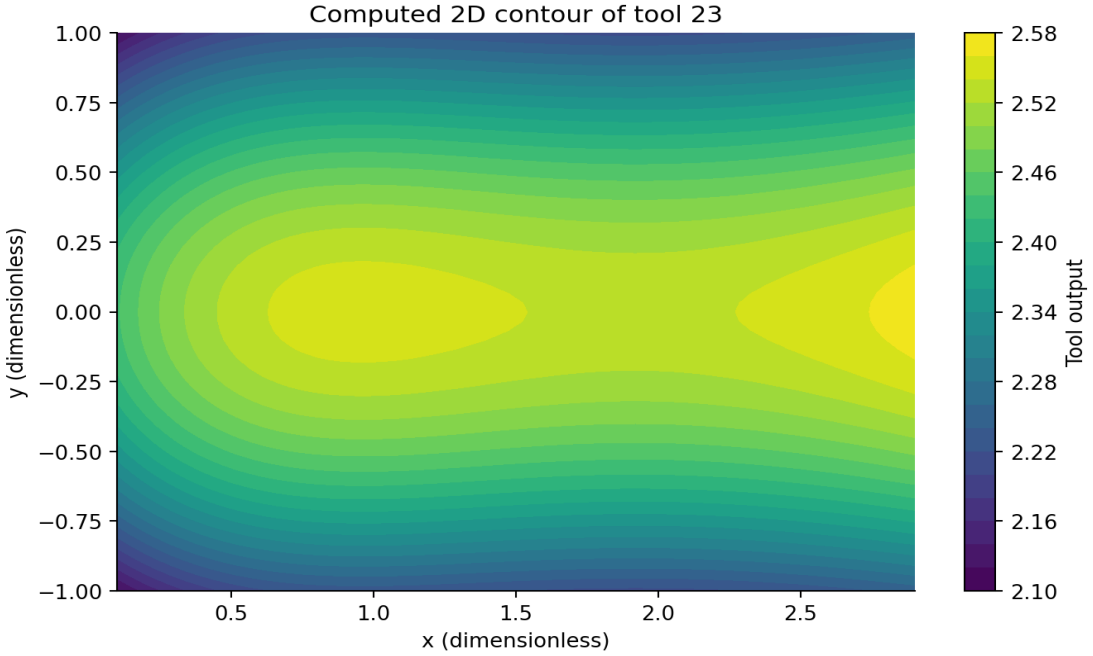


Figure 6. Contour map from the same exported 3,721 values.

8. Result distribution and implementation limits

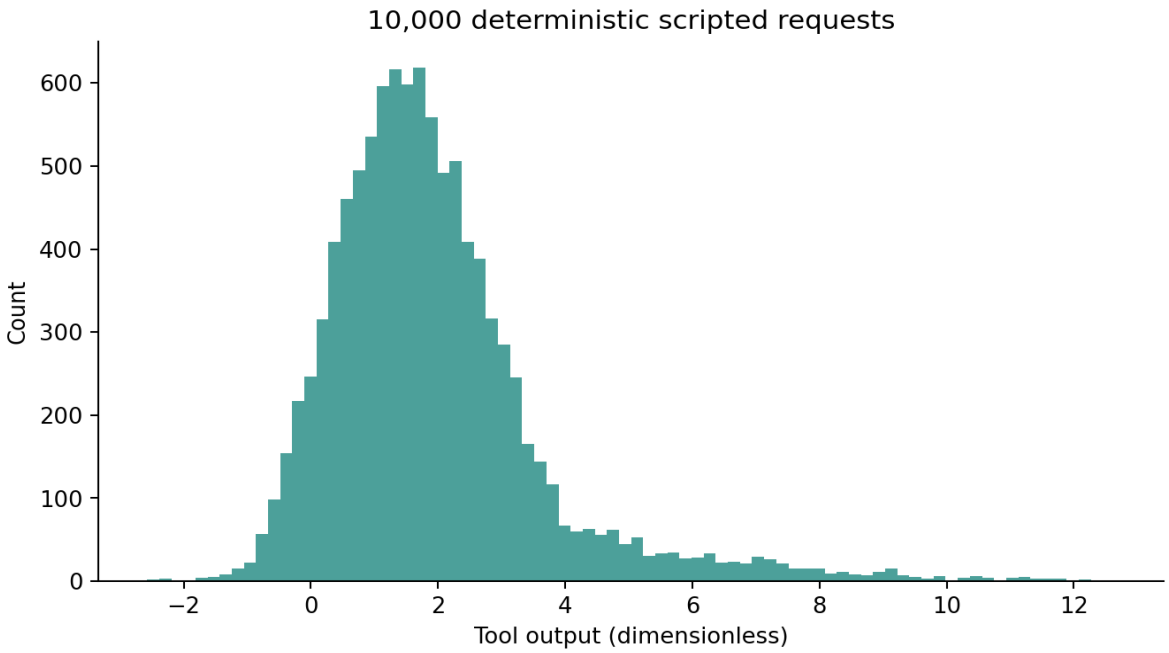


Figure 8. Distribution of the 10,000 scripted request outputs produced by the reusable paceval computation graphs.

Scientific and industrial limitations

The expressions use controlled dimensionless synthetic inputs. They are not calibrated material laws, approved finance models or deployed industrial digital twins. The software demonstrates reusable mathematical execution, graph versioning, and deterministic local tool chaining. It has not exercised networked MCP transport, AI selection correctness, real user input authorization, adversarial injection defense, service load balancing or production security controls.

The experiment does not use paceval Trusted Interval Computation, and it makes no interval certification claim. A production mathematical tool platform would additionally require schemas, safe resource budgets, validation, failure reporting, authentication, authorization and monitoring.

9. Reproduction and current technical references

Build and re-run

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release

-DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so

cmake --build build -j; ctest --test-dir build --output-on-failure

./build/quant_tools --out reproduced
```

The actual source code, model registry, 10,000-request CSV, 3,721-point surface, formula version audit, independent native C++ reference, 80-digit mpmath verification script and all generated scientific graphics are included. The original paceval C/C++ SDK facade is included under its original upstream notices; its external shared runtime is not redistributed.

Selected current sources

MCP tools specification (2026-07-28): https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/server/tools.mdx

MCP specification release (2026-07-28): https://blog.modelcontextprotocol.io/posts/2026-07-28/

MCP security and trust considerations: https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/index.mdx

paceval public API: https://paceval.com/api/

NIST AI Risk Management Framework: https://www.nist.gov/itl/ai-risk-management-framework

Final scientific conclusion

The defined 50-tool mathematical library has been successfully executed with 51 reusable paceval computation objects and independently verified on the stated synthetic workload. The demonstrated achievement is the numerical foundation of a future agent-accessible tool catalogue, not a verified production MCP server, an AI model or an operational industrial decision system.